

Capítulo tres

Análisis

El presente capítulo detallará el proceso de ingeniería aplicado para la construcción del sistema de monitoreo ambiental, la fase de este análisis constituye parte del proceso fundamental para poder traducir las necesidades de monitoreo ambiental del barrio “El Rosario” en especificaciones técnicas operativas y funcionales. Bajo el marco de trabajo de SCRUM, este proceso inicia con la identificación de los requerimientos críticos que permitan una captura precisa de contaminantes para el presente proyecto asegurando que la arquitectura de microservicios sea capaz de gestionar el flujo de datos desde los nodos sensores hasta la infraestructura en la nube de AWS.

El análisis no solo contempla la viabilidad técnica de la transmisión mediante el protocolo MQTT, sino que prioriza la experiencia del usuario final, garantizando que la información procesada y almacenada de forma híbrida en MongoDB e InfluxDB sea presentada de manera íntegra, segura y oportuna. Este diagnóstico inicial permite definir el Product Backlog, asegurando que cada funcionalidad desarrollada en los Sprints posteriores responda directamente a los objetivos de salud pública y transparencia informativa planteados en el proyecto.

3.1. Análisis de requisitos

El presente análisis define el “Qué” del sistema. Se identifican las necesidades de los usuarios finales y las restricciones técnicas impuestas por el entorno urbano y la infraestructura de red. Los requisitos funcionales describen las acciones específicas que el sistema debe realizar.

Tabla 10

Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción
RF-01	Adquisición de datos	El sistema debe leer niveles de material particulado y gases de forma continua.
RF-02	Transmisión inalámbrica	El nodo sensor debe enviar los datos al bróker MQTT mediante la red Wi-Fi local.
RF-03	Persistencia híbrida	El backend debe almacenar metadatos en

		MongoDB y series temporales en InfluxDB.
<i>RF-04</i>	Visualización en tiempo real	El Dashboard web debe mostrar gráficas dinámicas que se actualicen via WebSockets.
<i>RF-05</i>	Gestión de alertas	El sistema debe notificar visualmente cuando los niveles de contaminación superen los límites de la OMS.
<i>RF-06</i>	Autenticación de acceso	El acceso a la configuración del sistema debe estar protegido mediante tokens JWT.

Nota. Elaboración propia basado en los requerimientos funcionales del TIC

Los requisitos no funcionales representan las cualidades de calidad, rendimiento y seguridad que el sistema debe poseer. A continuación, en la tabla 11 se encuentran los requisitos no funcionales a considerar para el TIC:

Tabla 11

Requisitos No Funcionales (RF)

ID	Requisito No Funcional	Descripción
<i>RNF-01</i>	Disponibilidad	El sistema alojado en AWS debe garantizar una disponibilidad del 99.9%, permitiendo acceso 24/7.
<i>RNF-02</i>	Escalabilidad	La arquitectura debe permitir la adición de nuevos nodos sensores sin degradar el rendimiento.
<i>RNF-03</i>	Seguridad	Toda comunicación entre el cliente y el servidor debe estar cifrada mediante SSL/TLS
<i>RNF-04</i>	Latencia	El tiempo transcurrido desde la captura del sensor hasta la visualización en la web debe ser inferior a 1 segundo.
<i>RNF-05</i>	Bajo consumo	El firmware en C++ debe optimizar el uso de energía del microcontrolador para prolongar la vida útil del hardware.

Nota. Elaboración propia basado en los requerimientos no funcionales del TIC

3.1.2. Clasificación y Roles de los actores del sistema

A continuación, se presenta la matriz de actores identificados para el sistema de monitoreo ambiental del barrio “El Rosario”. Esta clasificación es la base para la configuración de la seguridad mediante JWT y la definición de las interfaces en React.js.

Tabla 12

Actores del sistema

ID	Descripción del Rol	Nivel de Acceso	Interés Principal
AC-01	Ciudadano / vecino: Habitante del sector que consume la información ambiental.	Público de solo lectura.	Salud personal y niveles de contaminación en tiempo real.
AC-02	Administrador: responsable técnico de la infraestructura informática.	Total, de lectura y escritura.	Estabilidad del sistema, gestión de la nube AWS y seguridad.

Nota. Elaboración propia basado en los actores identificados

3.1.3. Historias de Usuario

Las historias de usuario representan unidades de trabajo que describen una funcionalidad desde la perspectiva del actor final, bajo el marco de trabajo de SCRUM, estas historias nos permiten definir los criterios de aceptación técnicos para cada incremento del sistema. A continuación, en la tabla se detallan las historias que se consideraron prioridad para el sistema de monitoreo ambiental:

Tabla 12

Actores del sistema

ID	Actor	Historia de Usuario	Criterios de Aceptación
HU-01	Administrador	Como responsable técnico, quiero autenticarme mediante un sistema seguro para gestionar los nodos sensores sin riesgo de accesos no autorizados.	1. El inicio de sesión debe generar un token JWT. 1. El token debe expirar tras un periodo de inactividad para garantizar seguridad.
HU-02	Administrador	Como técnico, quiero monitorear el estado de conexión de los	1. Se debe mostrar un indicador de “En línea /

		dispositivos para identificar fallos en la red Wi-Fi de forma remota.	Fuera de Línea” basado en la telemetría de MongoDB. 2. El sistema debe registrar el último mensaje recibido del nodo.
<i>HU-03</i>	Administrador	Como responsable del sistema, quiero dar de alta o baja nuevos nodos sensores en el mapa del barrio de forma dinámica.	1. El sistema debe permitir registrar el ID del microcontrolador en MongoDB. 2. El nuevo nodo debe aparecer automáticamente en la interfaz web.
<i>HU-04</i>	Ciudadano	Como habitante del barrio, quiero visualizar los niveles de Dióxido de Carbono en tiempo real para conocer la calidad del aire antes de salir de casa.	1. La interfaz debe actualizarse mediante WebSockets de forma Automática. 2. Se debe mostrar una escala de colores según la normativa OMS.
<i>HU-05</i>	Ciudadano	Como usuario, quiero acceder a gráficas de la última semana para identificar las horas de mayor contaminación en la Avenida del Maestro.	1. El sistema debe consultar los datos en InfluxDB en menos de dos segundos. 2. Las gráficas deben ser interactivas y claras mostrando horarios de forma clara.
<i>HU-06</i>	Ciudadano	Como usuario, quiero recibir una notificación visual inmediata cuando la calidad del aire sea “Dañina” Según la OMS.	1. El dashboard debe cambiar de color y mostrar un mensaje de advertencia. 2. La alerta debe activarse sin que el usuario recargue la página.

Nota. Elaboración propia basado en las historias de usuario

3.1.4. Casos de Uso del Sistema

Los diagramas de casos de uso (UML) permiten modelar la funcionalidad del sistema centrándose en los servicios que este ofrece a sus diferentes actores. Dentro de la presente sección se describen los comportamientos del sistema de monitoreo para el barrio “El Rosario”, vinculando los requerimientos capturados en el análisis con la arquitectura técnica propuesta.

Dentro de la tabla 13 se consolidan todas las funcionalidades identificadas, categorizadas por el actor principal y el objetivo del negocio. Esta tabla nos permite observar la trazabilidad entre la necesidad del usuario y la respuesta técnica del sistema.

Tabla 13

Resumen general de Casos de Uso

ID	Módulo	Caso de Uso	Actor Principal	Funcionalidad
CU-01	Monitoreo	Visualizar monitoreo en Vivo.	Ciudadano	Observar niveles de CO_2 y O_3 .
CU-02	Monitoreo	Consultar Históricos	Ciudadano	Generar gráficas de tendencias temporales con consulta a través de InfluxDB
CU-03	Monitoreo	Interactuar con mapa de nodos	Ciudadano	Localización geográfica de los sensores dentro del área de recolección.
CU-04	Seguridad	Gestionar Sesión (Login).	Administrador	Validar identidad y obtener privilegios mediante tokens JWT.
CU-05	Seguridad	Recuperar contraseña.	Administrador	Flujo de seguridad para el restablecimiento de credenciales de

					acceso.
CU-06	Gestión	Registrar nuevo nodo sensor.	Administrador		Dar de alta nuevos microcontroladores insertando metadatos en MongoDB.
CU-07	Gestión	Configurar umbrales de alerta.	Administrador		Ajustar los rangos de toxicidad visual de acuerdo con normativa OMS.
CU-08	Mantenimiento	Calibrar sensor remotamente.	Administrador		Ajustar factores de escala en el hardware mediante mensajes MQTT.
CU-09	Mantenimiento	Monitorear salud del nodo.	Administrador		Revisar estado de conexión, batería y señal Wi-Fi en MongoDB.
CU-10	Mantenimiento	Reiniciar nodo remotamente.	Administrador		Envío de comando de reinicio al microcontrolador vía MQTT.
CU-11	Reportes	Auditar Logs de Actividad	Autoridad Ambiental		Revisar el Historial de cambios y accesos realizados en la plataforma.

Nota. Elaboración propia basado en los casos de uso

3.1.4.1. Casos de uso Específicos

A. CU-01:

Tabla 14

Caso de Uso CU-01

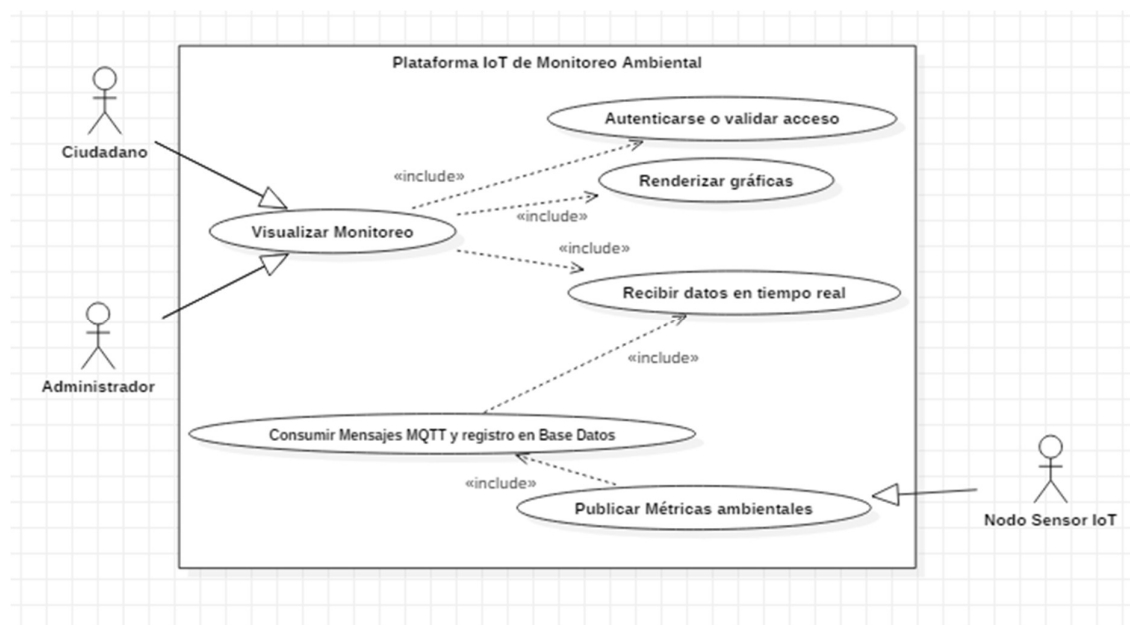
	Detalle
Identificador	CU-01
Nombre	Visualizar Monitoreo
Descripción	El sistema permite al ciudadano observar de forma reactiva los niveles de contaminación capturados a través de una conexión de datos persistentes.

<i>Precondición</i>	El nodo sensor está transmitiendo datos y el servicio de WebSockets en el servidor de AWS se encuentra activo.
<i>Postcondición</i>	El usuario visualiza cambios en las gráficas en milisegundos sin necesidad de interactuar con el navegador.
<i>Actores</i>	Ciudadano, Administrador, Nodo sensor
<i>Flujo de interacción</i>	• El ciudadano accede a la plataforma web cargada mediante el framework React.js. • El frontend solicita al backend (FastAPI) la apertura de un túnel de comunicación bidireccional mediante WebSockets. • El nodo sensor físico publica una trama JSON con las métricas ambientales hacia el Broker MQTT de AWS. • El backend consume el mensaje del Broker, valida su origen y lo registra de forma paralela en la base de datos. • El servidor empuja la información procesada a través del WebSocket activo hacia el cliente. • El frontend recibe el paquete de datos y actualiza instantáneamente los componentes visuales del dashboard.
<i>Comentario</i>	El flujo permite garantizar que exista una actualización en tiempo real, minimizando el consumo de recursos de red en el dispositivo del usuario.

Nota. Elaboración propia basado en el análisis del caso

Figura 14

Caso de Uso graficado CU-01



Nota. Adaptado de la tabla 14 y realizado con StarUML .CC BY 2.0.

B. CU-02:

Tabla 15

Caso de Uso CU-02

Detalle	
Identificador	CU-02
Nombre	Consultar históricos
Descripción	El sistema permite al usuario consultar y analizar el comportamiento pasado de las métricas ambientales mediante el uso de filtros temporales específicos.
Precondición	La base de datos de series temporales debe contar con registros previos almacenados para el periodo y los parámetros solicitados.
Postcondición	El usuario visualiza gráficas de tendencias históricas detalladas en la interfaz web para su análisis.
Actores	Ciudadano
Flujo de interacción	- El usuario selecciona el periodo de interés ya sea día, mes o semana y los parámetros específicos en el panel de filtros del dashboard. - El frontend envía una solicitud de consulta al backend con los parámetros temporales definidos por el usuario. - El backend procesa la petición y realiza una búsqueda optimizada en el

motor de base de datos externo de series temporales.

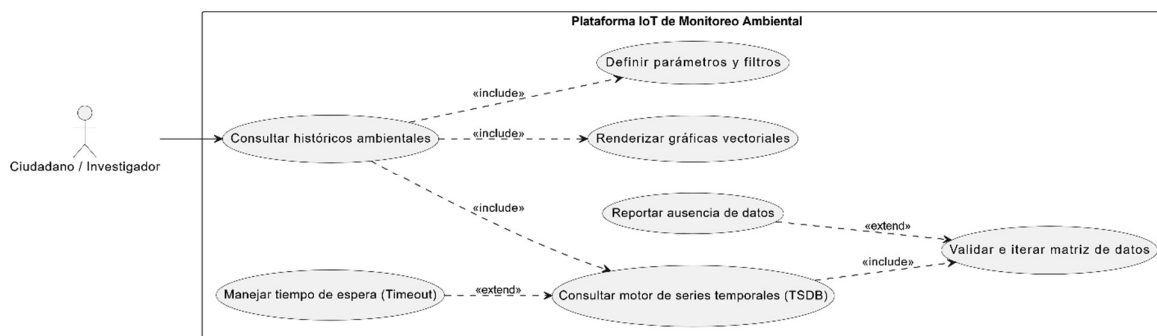
- La base de datos recupera los registros cronológicos correspondientes y los devuelve al servidor backend de forma estructurada.
- El backend valida la integridad de la información recibida y envía el paquete de datos procesados hacia el navegador del cliente.
- El frontend recibe la respuesta y renderiza las gráficas de tendencias históricas de forma interactiva en la pantalla del usuario.

Comentario El sistema debe estar capacitado para manejar volúmenes considerables de datos históricos sin comprometer la velocidad de respuesta de la interfaz.

Nota. Elaboración propia basado en el comportamiento a esperar de la base de datos dentro del caso

Figura 15

Caso de Uso graficado CU-02



Nota. Adaptado de la tabla 15 y hecho con StarUML. CC BY 2.0.

C. CU-03:

Tabla 16

Caso de Uso CU-03

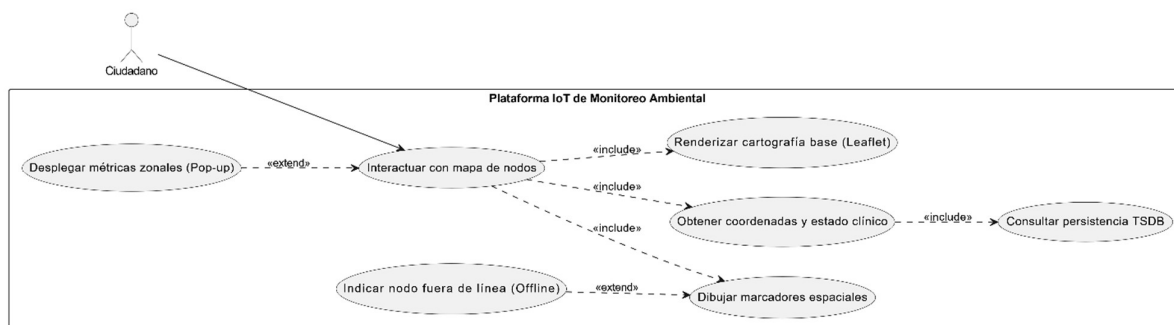
Detalle	
Identificador	CU-03
Nombre	Interactuar con mapa de nodos
Descripción	Permite al ciudadano visualizar la ubicación geográfica exacta de los puntos de monitoreo en el barrio “El Rosario” para identificar los focos de contaminación en su entorno cercano.
Precondición	Los metadatos de ubicación latitud y longitud del dispositivo debe estar correctamente registrado en la base de datos.
Postcondición	El usuario identifica geográficamente los sensores y accede a un resumen rápido de datos por zona.
Actores	Ciudadano

<i>Flujo de interacción</i>	- El ciudadano selecciona la sección de “Mapa de Red” dentro de la interfaz web desarrollada en React.js. - El frontend realiza una petición al backend para obtener la lista de coordenadas y el estado de salud de todos los nodos desplegados. - El backend consulta en la base de datos externa InfluxDB los registros de ubicación y los identificadores únicos de cada microcontrolador. - El sistema de base de datos devuelve la información geográfica y el servidor la envía a la cliente debidamente estructurada. - El frontend utiliza la librería Leaflet para renderizar los marcadores interactivos sobre la capa de mapas. - El usuario interactúa con un marcador específico para desplegar una ventana emergente que muestra el nombre del nodo y el último índice de calidad del aire registrado.
<i>Comentario</i>	El mapa utilizará una codificación de colores en los marcadores para indicar si un nodo está activo o fuera de línea.

Nota. Elaboración propia basado en la posible interacción de los nodos.

Figura 16

Caso de Uso graficado CU-03.



Nota. Adaptado de la tabla 16 y hecho con StarUML. CC BY 2.0.

D. CU-04:

Tabla 17

Caso de Uso CU-04

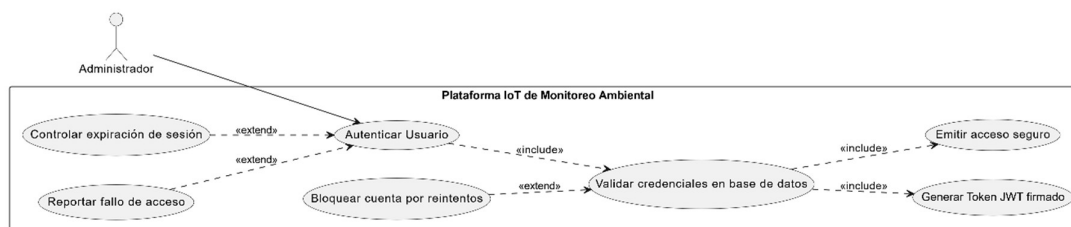
	Detalle
<i>Identificador</i>	CU-04
<i>Nombre</i>	Gestionar Sesión
<i>Descripción</i>	Permite la validación de identidad de los usuarios administrativos para habilitar el acceso a las funciones de gestión y mantenimiento de la red de sensores.

<i>Precondición</i>	Las credenciales del usuario deben estar registradas previamente en el motor de base de datos NoSQL del sistema.
<i>Postcondición</i>	El sistema otorga un token de seguridad JWT que permite el acceso a funcionalidades y rutas protegidas.
<i>Actores</i>	Administrador
<i>Flujo de interacción</i>	- El usuario ingresa sus credenciales de acceso como correo y contraseña en el formulario de inicio de sesión de la plataforma. - El frontend captura la información y la envía de forma segura mediante una petición cifrada hacia el servidor backend. - El backend consulta en la base de datos externa InfluxDB para poder verificar la existencia del usuario y la validez de su contraseña. - Tras validar exitosamente lo anterior, el servidor genera un token JWT firmado digitalmente que contiene el rol y el tiempo de expiración. - El sistema envía el token de seguridad de vuelta al navegador del usuario para su almacenamiento en estado local. - El frontend reconoce el token valido y habilita automáticamente las herramientas administrativas correspondientes al perfil del usuario.
<i>Comentario</i>	El sistema denegará el acceso y notificará al usuario en caso de que las credenciales no coincidan con los registros en la base de datos.

Nota. Elaboración propia basado en la administración del login

Figura 17

Caso de Uso graficado CU-04



Nota. Adaptado de la tabla 17 y hecho con StarUML. CC BY 2.0.

E . CU-05:

Tabla 18

Caso de Uso CU-05

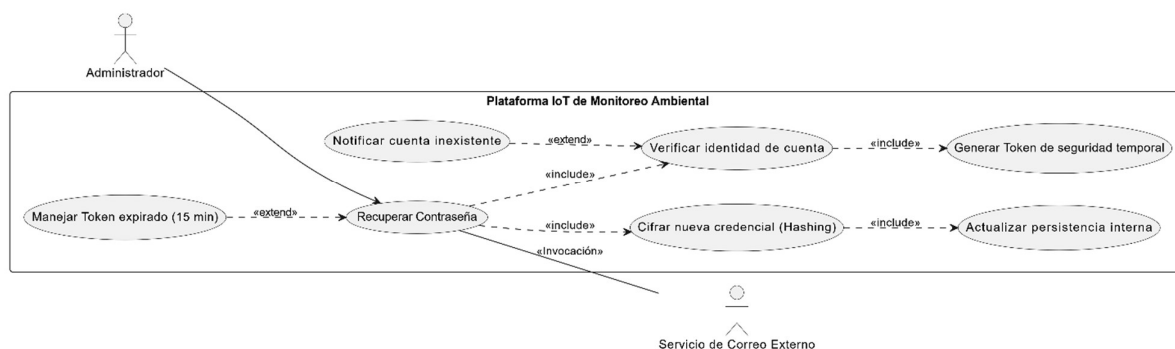
	Detalle
<i>Identificador</i>	CU-05
<i>Nombre</i>	Recuperar Contraseña
<i>Descripción</i>	Permite a los usuarios con roles administrativos restablecer sus credenciales de

	acceso de forma segura en caso de olvido o pérdida.
<i>Precondición</i>	El correo electrónico del usuario debe estar registrado y activo en la base de datos externa InfluxDB.
<i>Postcondición</i>	El sistema actualiza el hash de la contraseña en la base de datos y el usuario recupera el acceso a sus funciones.
<i>Actores</i>	Administrador, Servicio de correo.
<i>Flujo de interacción</i>	• El usuario ingresa su correo electrónico registrado en el formulario de recuperación de la interfaz web en React.js. • El backend recibe la solicitud y verifica la existencia de la cuenta en la base de datos externa InfluxDB. • El sistema genera un token de recuperación único y temporal, asociándolo a la cuenta del usuario por un tiempo limitado. • Se invoca a un servicio externo de mensajería para enviar un correo electrónico al usuario con un enlace de redirección seguro. • El usuario accede al enlace, ingresa su nueva contraseña y el backend procede a cifrarla mediante algoritmos de hashing. • El sistema sobre escribe la credencial antigua en InfluxDB, invalida el token de recuperación y confirma el éxito del proceso en la interfaz.
<i>Comentario</i>	El enlace de recuperación tendrá una vigencia de 15 minutos; transcurrido ese tiempo, el sistema anulará el proceso por motivos de seguridad.

Nota. Elaboración propia basado en la posible interacción al recuperar contraseña

Figura 18

Caso de Uso graficado CU-05



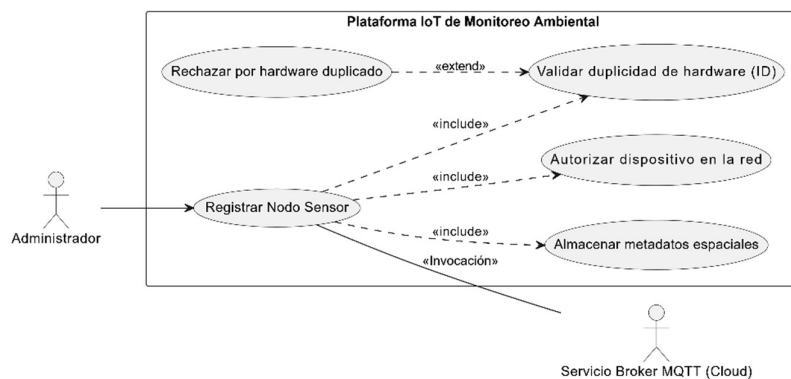
Nota. Adaptado de la tabla 18 y hecho con StarUML. CC BY 2.0.

F. CU-06:**Tabla 19***Caso de Uso CU-06*

Detalle	
<i>Identificador</i>	CU-06
<i>Nombre</i>	Registrar Nodo Sensor
<i>Descripción</i>	Permite al administrador dar de alta un nuevo dispositivo físico en la plataforma, vinculando su identidad técnica con su ubicación geográfica en el barrio.
<i>Precondición</i>	El administrador debe contar con una sesión activa (JWT) y poseer el identificador único ID o dirección MAC del microcontrolador.
<i>Postcondición</i>	El nuevo nodo queda habilitado en la base de datos para comenzar a recibir y procesar sus tramas de datos ambientales.
<i>Actores</i>	Administrador, AWS Cloud.
<i>Flujo de interacción</i>	• El administrador ingresa al módulo de gestión de dispositivos y completa el formulario con el ID del sensor, descripción y coordenadas de instalación. • El frontend en React.js valida el formato de los datos y envía una petición de registro hacia el backend del sistema. • El backend (FastAPI) verifica que el identificador del hardware no exista previamente para evitar duplicidad en la red. • El sistema interactúa con la base de datos externa InfluxDB para insertar el nuevo documento con los metadatos y el estado inicial del dispositivo. • El backend comunica al servicio de mensajería en la nube la autorización del nuevo ID para permitir futuras publicaciones vía protocolo MQTT. • El sistema confirma la creación exitosa en la interfaz y actualiza automáticamente el listado global de nodos activos en el dashboard.
<i>Comentario</i>	Una vez registrado, el nodo pasará al estado “pendiente” hasta que el hardware realice su primera transmisión hacia la nube de forma exitosa.
<i>Nota.</i> Elaboración propia basado en el flujo para añadir un nuevo nodo sensor	

Figura 19

Caso de Uso graficado CU-06



Nota. Adaptado de la tabla 19 y diseñado con StarUML. CC BY 2.0.

G. CU-07:

Tabla 20

Caso de Uso CU-07

Detalle	
Identificador	CU-07
Nombre	Configurar Umbrales de alerta
Descripción	Permite definir y actualizar los rangos de concentración de contaminantes ($PM_{2.5}$ y O_3) que disparan cambios visuales y alertas en la plataforma, basándose en normativas de salud.
Precondición	El administrador debe contar con una sesión activa y privilegios de escritura en la base de datos.
Postcondición	Los nuevos parámetros se aplican globalmente a la lógica de visualización y al motor de notificaciones del sistema.
Actores	Administrador
Flujo de interacción	- El administrador accede al módulo de “Configuración de normativa” en el panel administrativo de la web. - El sistema recupera los valores de umbrales actuales desde la colección de configuración en la base de datos InfluxDB. - El administrador modifica los límites de toxicidad, por ejemplo, niveles de “Bueno”, “moderado”, “Dañino” siguiendo las guías actualizadas de la OMS. - El frontend envía la nueva matriz de parámetros al backend, el cual valida que los rangos sean lógicos y numéricamente consistentes. - El backend actualiza el documento de configuración en MongoDB y emite

una señal de refresco de caché para los servicios de monitoreo.

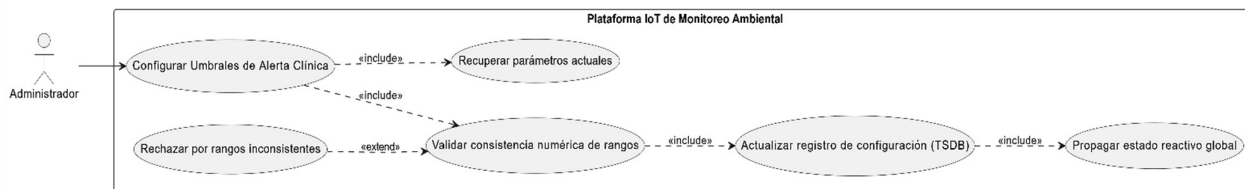
- El sistema confirma la actualización exitosa y ajusta automáticamente el color de los indicadores en el dashboard de todos los usuarios activos.

Comentario El sistema debe mantener un registro de la versión de la normativa aplicada para permitir la trazabilidad histórica de los cambios de criterio en las alertas.

Nota. Elaboración propia basado en el flujo de configuración para los umbrales de alerta

Figura 20

Caso de Uso graficado CU-07



Nota. Adaptado de la tabla 20 y diseñado con StarUML. CC BY 2.0.

H. CU-08:

Tabla 21

Caso de Uso CU-08

	Detalle
Identificador	CU-08
Nombre	Calibrar Sensor Remotamente
Descripción	Permite ajustar los parámetros de conversión analógico-digital del hardware físico para corregir desviaciones en las lecturas de gas y partículas.
Precondición	El técnico debe tener una sesión activa y el Nodo Sensor debe estar suscrito al tópico de control en el Broker MQTT de AWS.
Postcondición	El microcontrolador actualiza su lógica de cálculo en tiempo real y el sistema registra la nueva configuración en la base de datos.
Actores	Administrador, Nodo Sensor, AWS Broker MQTT
Flujo de interacción	• El Administrador ingresa los nuevos coeficientes de calibración para el sensor específico a través del panel técnico en la interfaz de React.js. • El backend valida la consistencia de los parámetros y genera un comando estructurado bajo el protocolo de mensajería del sistema. • El sistema publica el comando de actualización en el tópico de suscripción del hardware mediante el Broker MQTT alojado en la infraestructura de AWS. • El nodo Sensor recibe la instrucción de forma inalámbrica, actualiza los valores en su memoria volátil y ajusta inmediatamente su algoritmo de

muestreo.

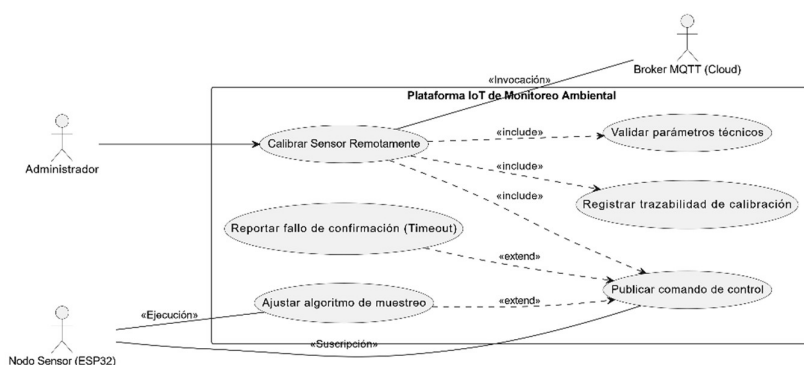
- El hardware envía un mensaje de confirmación ACK hacia el servidor para validar que la nueva configuración ha sido aplicada correctamente.
- El backend registra la fecha y los valores de la nueva calibración en InfluxDB para poder mantener la trazabilidad y notifica el éxito de la operación en el frontend.

Comentario Si el nodo no confirma la recepción en un tiempo determinado, el sistema alertará al técnico sobre un posible fallo de comunicación o latencia exclusiva.

Nota. Elaboración propia basado en el flujo de calibración remota

Figura 21

Caso de Uso graficado CU-08



Nota. Adaptado de la tabla 21 y diseñado con StarUML. CC BY 2.0.

3. CU-09:

Tabla 22

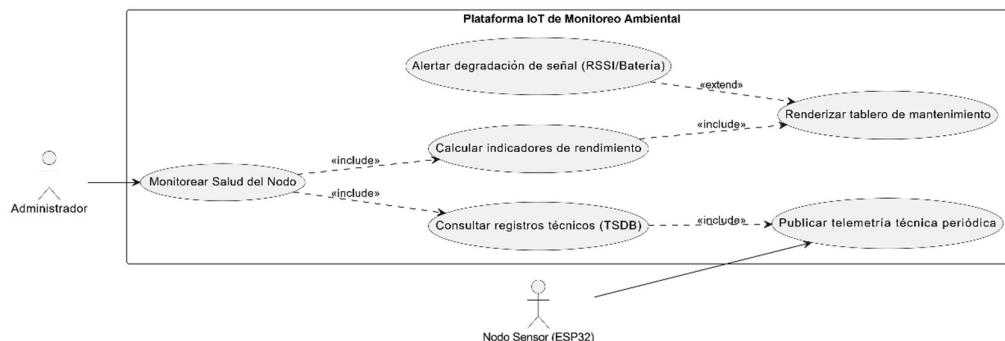
Caso de Uso CU-09

Detalle	
Identificador	CU-09
Nombre	Monitorear Salud del Nodo
Descripción	Permite al técnico supervisar el estado operativo del hardware, analizando variables críticas como la intensidad de señal Wi-Fi, el nivel de batería y el tiempo de actividad.
Precondición	Los nodos sensores deben estar enviando paquetes de telemetría técnica de forma periódica a la infraestructura de red.
Postcondición	El técnico obtiene un diagnóstico preciso del estado físico y lógico de cada punto de monitoreo en el barrio.
Actores	Administrador, Nodo Sensor
Flujo de	• El Administrador accede al panel de "Mantenimiento técnico" dentro de la

<i>interacción</i>	plataforma web en React.js. • El frontend solicita al backend la última ráfaga de datos técnicos y telemetría de todos los dispositivos registrados. • El backend realiza una consulta a la base de datos externa InfluxDB para extraer los valores de señal, voltaje y última conexión. • El sistema de base de datos devuelve los registros de salud técnica ordenados por marca de tiempo hacia el servidor. • El backend procesa los datos y calcula los indicadores de rendimiento, por ejemplo, el porcentaje de paquetes perdidos o degradación de señal. • El sistema renderiza en la interfaz una serie de indicadores visuales
<i>Comentario</i>	El sistema resaltará en color ámbar o rojo aquellos nodos cuyo nivel de batería o señal Wi-Fi se encuentre por debajo de los umbrales operativos mínimos.
<i>Nota.</i> Elaboración propia basado en el flujo de calibración remota	

Figura 22

Caso de Uso graficado CU-09

*Nota.* Adaptado de la tabla 22 y diseñado con StarUML. CC BY 2.0.**J. CU-10:****Tabla 23**

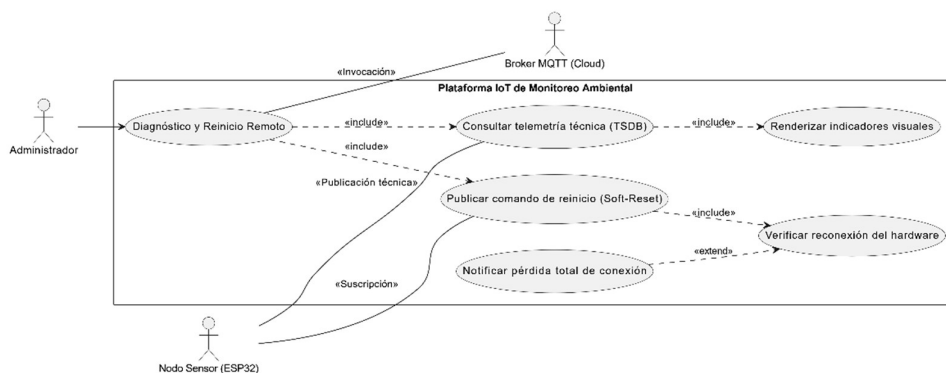
Caso de Uso CU-10

Detalle	
<i>Identificador</i>	CU-10
<i>Nombre</i>	Reiniciar Nodo Remotamente
<i>Descripción</i>	Permite enviar una instrucción de reinicio lógico al microcontrolador para restaurar su estado operativo en caso de latencia alta o errores de ejecución.
<i>Precondición</i>	Los nodos sensores deben estar enviando paquetes de telemetría técnica de forma periódica a la infraestructura de red.

<i>Postcondición</i>	El técnico obtiene un diagnóstico preciso del estado físico y lógico de cada punto de monitoreo en el barrio.
<i>Actores</i>	Administrador, Nodo Sensor, Broker MQTT
<i>Flujo de interacción</i>	- El administrador accede al panel de “Mantenimiento técnico” dentro de la plataforma web en React.js. - El frontend solicita al backend la última ráfaga de datos técnicos y telemetría de todos los dispositivos registrados. - El backend realiza una consulta a la base de datos externa InfluxDB para extraer los valores de señal, voltaje y última conexión. - El sistema de base de datos devuelve los registros de salud técnica ordenados por marca de tiempo hacia el servidor. - El backend procesa los datos y calcula los indicadores de rendimiento, por ejemplo, el porcentaje de paquetes perdidos o degradación de señal. - El sistema renderiza en la interfaz una serie de indicadores visuales
<i>Comentario</i>	El sistema resaltará en color ámbar o rojo aquellos nodos cuyo nivel de batería o señal Wi-Fi se encuentre por debajo de los umbrales operativos mínimos.
<i>Nota.</i>	Elaboración propia basado en el reinicio remoto planeado para el nodo

Figura 23

Caso de Uso graficado CU-10



Nota. Adaptado de la tabla 23 y diseñado con StarUML. CC BY 2.0.

K. CU-11:

Tabla 25

Caso de Uso CU-11

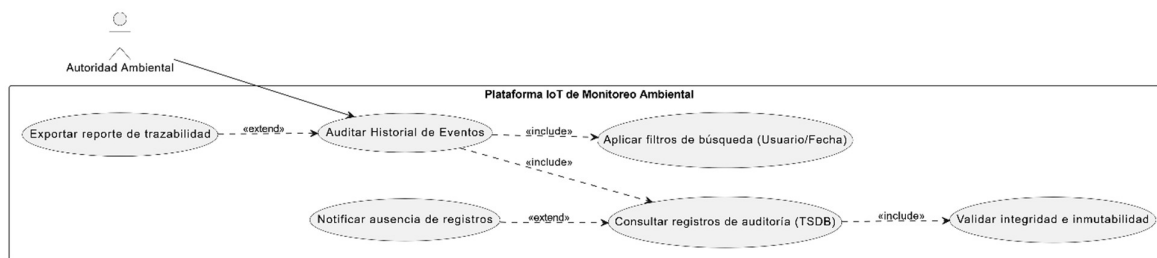
Detalle	
<i>Identificador</i>	CU-11

Nombre	Auditar Logs
Descripción	Permite la supervisión y revisión de la bitácora de eventos para identificar qué usuarios realizaron cambios críticos en el sistema (Registros, calibraciones o reinicios).
Precondición	El Usuario debe tener el rol de “Autoridad Ambiental” y el sistema debe haber registrado previamente las acciones en la colección de auditoría.
Postcondición	El analista obtiene una visión transparente y cronológica de la operatividad administrativa de la plataforma.
Actores	Administrador, Autoridad Ambiental
Flujo de interacción	• El administrador ingresa al módulo de “Auditoría de Sistema” en la interfaz de usuario para revisar la trazabilidad de las acciones. • El frontend solicita al servidor la lista de eventos, permitiendo filtrar por usuario, tipo de acción o rango de fechas. • El backend realiza una consulta a la colección de logs en la base de datos externa InfluxDB para extraer los registros de actividad. • El motor de la base de datos devuelve los documentos que incluyen el ID del usuario, la marca de tiempo, la dirección IP y la descripción de la acción realizada. • El backend procesa la información y la envía al cliente asegurando que la data sea inmutable y esté debidamente formateada para su lectura. • El frontend renderiza una tabla interactiva que permite a la autoridad inspeccionar cada evento ocurrido en la infraestructura.
Comentario	El sistema garantizará que los registros de auditoría no puedan ser modificados ni eliminados por ningún usuario, cumpliendo con estándares de seguridad de la información.

Nota. Elaboración propia basado en el análisis del flujo en la exportación de datos auditables

Figura 24

Caso de Uso graficado CU-11



Nota. Adaptado de la figura 24 y diseñado con StarUML. CC BY 2.0.

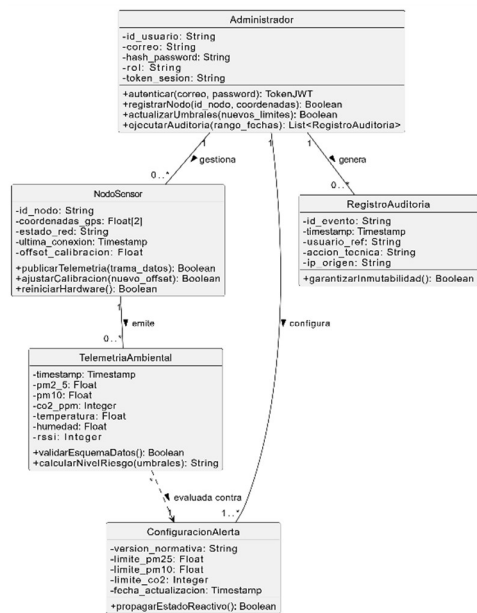
3.4.1.2. Diagrama de Clases

Para abstraer la estructura estática del sistema, se realiza el siguiente diseño del diagrama de clases enfocado en el modelo de dominio. En el contexto de la arquitectura backend planteada a desarrollar en Python, estas clases representan los esquemas de validación de datos (Data Transfer Objects) y las entidades lógicas que interactúan con el motor de InfluxDB y Broker MQTT.

En la figura 26 se ilustra las propiedades, métodos y multiplicidades de las entidades centrales de la investigación epidemiológica.

Figura 25

Diagrama de Clases



Nota. figura 25 en base al diseño propuesto y diseñado con StarUML. CC BY 2.0.

3.1.5. Planificación de Sprints

La implementación del sistema se divide en tres iteraciones (Sprints), priorizando la infraestructura técnica antes que la estética visual.

Sprint 1: Infraestructura de Red, Seguridad y Gestión de Nodos

Objetivo: Establecer la base de comunicación segura entre el hardware y la nube de AWS, habilitando el control administrativo. Dentro del presente sprint estarán proyectadas las tareas relacionadas con las historias de usuario: HU-01, HU-02, y HU-03. En la tabla 26 se pueden ver el orden y clasificación de tareas estimando su prioridad y tiempo en realizar.

Tabla 26

Sprint 1

Código	Actividad / Tarea	Prioridad	Estimación
T1-1	Configuración del entorno de backend (FastAPI) y conexión con InfluxDB.	Alta	5 días
T1-2	Desarrollo del servicio de autenticación y generación de tokens JWT (HU-01).	Alta	4 días
T1-3	Configuración del Broker AWS IoT Core y creación de políticas de seguridad.	Alta	3 días
T1-4	Implementación del firmware base en el ESP32 para el envío de telemetría (HU-02).	Media	6 días
T1-5	Diseño de la interfaz de registro de nodos y almacenamiento de metadatos (HU-03).	Media	4 días

Nota. Elaboración propia basado en el análisis de las tareas técnicas necesarias para iniciar el primer sprint

Sprint 2: Monitoreo en Tiempo Real y Lógica de Alertas

Objetivo: Desarrollar el flujo de datos dinámico y la interpretación de calidad de aire según la OMS. En este segundo Sprint se encuentran presentes las tareas relacionadas con las historias de usuario HU-05. HU-07. Dentro de la tabla 27 se pueden encontrar la clasificación de dichas tareas.

Tabla 27*Sprint 2*

Código	Actividad / Tarea	Prioridad	Estimación
T2-1	Implementación de WebSockets en el servidor para transmitir datos en tiempo real (HU-05).	Alta	5 días
T2-2	Creación del Dashboard principal en React.js con indicadores reactivos.	Alta	7 días
T2-3	Desarrollo de la lógica de negocio para escala de colores y alertas de la OMS (HU-07).	Alta	3 días
T2-4	Configuración de las reglas de AWS IoT para la persistencia de datos en InfluxDB.	Media	4 días
T2-5	Pruebas de Integración Hardware-Nube-Frontend.	Media	3 días

Nota. Elaboración propia basado en el análisis de las tareas lógicas de desarrollo para continuar la implementación en el TIC

Sprint 3: Monitoreo en Tiempo Real y Lógica de Alertas

Objetivo: Finalizar las herramientas de análisis a largo plazo y la capacidad de exportación legal de datos. En el tercer y último sprint encontramos todas las tareas relacionadas con las historias de usuario: HU-04 y HU-06, dentro de la tabla 28 se pueden contemplar las actividades a realizar para cumplir dichas historias.

Tabla 28*Sprint 3*

Código	Actividad / Tarea	Prioridad	Estimación
T3-1	Desarrollo de consultas analíticas sobre InfluxDB para gráficas temporales (HU-06).	Alta	5 días
T3-2	Integración del mapa georreferenciado con Leaflet para ubicación de nodos.	Media	4 días
T3-3	Optimización de tiempos de respuesta en la API para	Baja	3 días

	consultas masivas.		
T3-4	Despliegue final de la solución en infraestructura Cloud y documentación técnica.	Alta	4 días

Nota. Elaboración propia basado en el análisis de las actividades finales a realizar en el tercer sprint

3.1.6. Planificación en base al análisis

Tras el levantamiento de requerimientos y la definición de las Historias de Usuario y Casos de Uso, se determinó que la ejecución del presente proyecto de titulación no puede ser lineal, sino incremental. La planificación se fundamenta en la ruta crítica de datos: desde la percepción físicas (Sensores) hasta el punto de persistencia y visualización.

3.1.6.1. Priorización de requerimientos

Para asegurar un Producto Mínimo Viable funcional desde las primeras etapas, se aplicó un criterio de priorización basado en la dependencia técnica, siguiendo la lógica descrita a continuación:

- **Capa de conectividad:** Es crítico ya que sin una comunicación entre el ESP32 y el Broker AWS IoT Core, no existe flujo de información, por ello, el registro de nodos y la telemetría técnica tienen una prioridad máxima.
- **Capa de seguridad y persistencia:** La gestión de sesiones mediante JWT y la estructuración de base de datos (InfluxDB) deben estar listas antes de cualquier intento de visualización para garantizar la integridad de los datos.
- **Capa de presentación:** La interfaz de usuario en React.js se desarrolla sobre servicios ya probados, asegurando que el ciudadano reciba información veraz y en tiempo real.

3.1.6.2. Estrategia de Mitigación de Riesgos Técnicos

Considerando el análisis previo, se identificaron los siguientes riesgos que dictan el orden de la planificación, estos riesgos se encuentran listados en la tabla 29:

Tabla 29

Riesgos de planificación

Riesgo Identificado	Impacto	Estrategia de Mitigación
<i>Inestabilidad de Red Wi-Fi</i>	Alto	Implementación de Reinicio Remoto (CU-10) y monitoreo de señal en el Sprint 1.
<i>Saturación de InfluxDB</i>	Medio	Diseño de consultas agregadas y límites de exportación (CU-11) desde la fase de desarrollo.
<i>Latencia en datos en Vivo</i>	Medio	Uso de WebSockets para evitar el refresco constante de la página y optimizar el consumo de recursos.

Nota. Elaboración propia basado en el análisis de las actividades finales a realizar en el tercer sprint

3.1.6.3. Estimación de Recursos y Esfuerzo

Para garantizar la viabilidad del proyecto de titulación relacionado con el monitoreo ambiental en el Barrio “El Rosario”, se ha realizado una cuantificación detallada de los elementos necesarios. Esta estimación no solamente contempla los componentes físicos, sino el esfuerzo intelectual y tecnológico requerido para la integración de la infraestructura IoT.

I. **Talento Humano y Esfuerzo de Desarrollo**, el proyecto es ejecutado por un único investigador, quien asume roles multidisciplinarios. El esfuerzo se calcula en base a una jornada estimada de 20 horas semanales durante un periodo de 12 semanas (un total de 240 horas-hombre), distribuidas de la siguiente manera:

- **Analista de sistemas (15%):** levantamiento de requerimientos, definición de historias de usuario y diagramación UML.
- **Desarrollador de firmware (30%):** programación de microcontroladores ESP32, gestión de protocolos MQTT y calibración de sensores de CO_2 y $PM_{2.5}$.
- **Ingeniero Cloud/Backend (30%):** configuración de la infraestructura en AWS, gestión de base de datos NoSQL y seguridad JWT.
- **Desarrollador Frontend (25%):** Diseño de la interfaz en React.js, integración de mapas y visualización de datos en tiempo real.

II. **Recursos de Hardware para la capa de Percepción y Desarrollo**, se detallarán únicamente los insumos físicos críticos para la captura de datos y el entorno de construcción:

- **Dispositivos finales:** Microcontroladores ESP32 con Módulos Wi-Fi integrados.
- **Instrumentación:** sensores de partículas finas $PM_{2.5}$, sensores Infrarrojos de CO_2 y módulos de gestión de energía.
- **Hardware de desarrollo:** Laptop con capacidad de virtualización para contenedores y compilación de firmware.

III. **Recursos de Software y Servicios Cloud**, la arquitectura se apoya en herramientas de código abierto y servicios bajo demanda para minimizar costos iniciales y maximizar la escalabilidad:

- **Entornos de desarrollo:** Visual Studio Code (IDE), Arduino IDE, Postman para pruebas de API.
- **Gestión de Datos:** MongoDB Atlas para la persistencia documental e InfluxDB Cloud para el manejo de series temporales de alta frecuencia.
- **Plataformas IoT:** AWS IoT Core para el manejo de sombras de dispositivos (Device Shadows) y enrutamiento de mensajes.

IV. Resumen de Carga de Trabajo por Fase

En la tabla 30 se puede reflejar de manera resumida y concreta la carga de trabajo por cada fase del proyecto.

Tabla 30

Carga de trabajo por fases

Fase del Proyecto	Esfuerzo (HH)	Nivel de Complejidad	Entregable Principal
Análisis Requisitos	y 36h	Media	Documentos de especificación
Desarrollo	de 72h	Alta	Código fuente del ESP32

<i>Firmware</i>			
<i>Infraestructura y API</i>	72h	Alta	Servicios Cloud y Endpoints
<i>Interfaz y Reportes</i>	60h	Media	Aplicación Web Funcional

Nota. Elaboración propia basado en el desglose de las fases del proyecto

Capítulo Cuatro

Desarrollo de la Solución

El presente capítulo detalla la fase de construcción e implementación de la red de monitoreo epidemiológico. Para poder garantizar la trazabilidad y calidad del software a construir, la construcción se rige bajo una metodología ágil iterativa, dividida en ciclos de desarrollo (Sprints). El proceso inicia con un Sprint 0 fundamental destinado a la arquitectura estructural, seguido de Sprints incrementales donde se codifican, prueban y validan las Historias de Usuario definidas en el marco de los requisitos funcionales.

4.1. Sprint 0: Diseño de la solución (Modelo C4)

El Sprint 0 constituye la fase de cimentación técnica del proyecto. Para modelar la arquitectura de la solución, se adopta el estándar C4 (Contexto, Contenedores, Componentes y Código). Este enfoque jerárquico permite abstraer la complejidad del sistema IoT, ofreciendo diferentes niveles de profundidad visual, desde la interacción que tendrá el ciudadano con el ecosistema global hasta las rutinas internas del servidor Backend.

4.1.1. Diseño lógico de la Base de Datos (TSBD)

A diferencia de los sistemas de información transaccionales que utilizan esquemas relacionales SQL o colecciones de documentos NoSQL, la naturaleza clínica de este proyecto exige el registro ininterrumpido de telemetría ambiental de alta frecuencia. Por consiguiente, el diseño lógico de la persistencia se estructura bajo el paradigma de una Base de Datos de Series Temporales, utilizando el motor InfluxDB. El esquema lógico no se organiza en tablas, sino en un modelo de protocolo de línea (Line Protocol) compuesto por cuatro elementos estructurales: Measurement (tabla virtual), Tags (Metadatos Indexados